CAPA

Computational Architecture for Predicting Alloimmunity

Technical Debrief — Version 1.0

April 2026

Huanxuan (Shawn) Li

Thomas Jefferson High School for Science & Technology

Repository	github.com/sh4wn27/capa
Language	Python 3.11+ / TypeScript (web)
Framework	PyTorch 2.x · FastAPI · Next.js 14
Dataset	UCI BMT Children (n=187) · IPD-IMGT/HLA
Status	Prototype — baselines evaluated, full ESM-2 pipeline implemented

1. Executive Summary

CAPA is an open-source deep learning framework designed to replace categorical HLA match/mismatch counts in haematopoietic stem cell transplantation (HSCT) outcome prediction with **continuous, structure-aware protein language model embeddings**. It models the three dominant post-transplant competing events — acute graft-versus-host disease (GvHD), disease relapse, and transplant-related mortality (TRM) — jointly through a DeepHit survival head, yielding calibrated cumulative incidence functions (CIFs) over a 730-day post-transplant horizon.

Version 1 represents a complete prototype: the full Python package is implemented and tested across all modules, a FastAPI inference backend is functional, and a Next.js 14 web interface provides an interactive prediction tool. Three tabular baseline models (cause-specific Cox, Fine-Gray subdistribution hazard, and a flat-feature DeepHit MLP) have been trained and evaluated on the publicly available UCI BMT paediatric dataset (n=187). The **primary scientific gap** is the absence of allele-level HLA typing in the UCI dataset, which prevents end-to-end validation of the ESM-2 embedding pipeline on real patient data.

Component	Status	Notes
Data pipeline	Complete	Loader, parser, splits, feature engineering
ESM-2 embedder	Complete	CPU/CUDA/MPS; HDF5 cache; 98-allele prototype
Cross-attention	Complete	2 layers, 8 heads, d'=128, attention export
DeepHit head	Complete	Joint PMF + CIF; cause-specific head also available
Training loop	Complete	AdamW, cosine LR, early stopping, checkpointing
Evaluation	Complete	C-index, IBS, calibration, 1000-iter bootstrap CI
Baseline models	Complete	Cox-CS 0.75/0.65, Fine-Gray 0.84/0.66, DH 0.67/0.41
FastAPI backend	Complete	Mock fallback when no checkpoint; CORS configured
Web frontend	Complete	Next.js 14; RiskChart; AttentionHeatmap; HLACombobox
configs/ YAML	Missing	scripts/train.py crashes without configs/default.yaml
Model checkpoint	Missing	No trained .pt file; backend serves mock responses
Backend deployment	Missing	CAPA_BACKEND_URL unset in Vercel; no Modal/Railway setup
Notebooks 01,03,04	Missing	Only 02_embeddings.ipynb present in repo

Table: Version 1 completion status across all system components

2. Scientific Motivation

Allogeneic HSCT remains the only curative option for many haematological malignancies, with over 50,000 procedures performed globally per year. Five-year overall survival for unrelated-donor transplants remains below 50% across many disease categories. Donor selection — the primary lever available to clinicians before transplant — depends critically on HLA compatibility assessment.

2.1 The Categorical HLA Scoring Problem

Current clinical practice evaluates HLA compatibility by counting mismatched loci. A '10/10 matched' unrelated donor means agreement at HLA-A, -B, -C, -DRB1, and -DQB1 (two alleles each). Each additional mismatch increments the count. This representation has two fundamental limitations:

- **All mismatches treated as equivalent.** An A*02:01 vs A*03:01 mismatch (differing at >15 residues in the peptide-binding groove) is penalised identically to an A*02:01 vs A*02:06 mismatch (differing at 2 residues). The immunological consequence is wildly different.
- **Directional and locus-specific information is lost.** Aggregate mismatch counts collapse five loci into a single integer, discarding which loci are mismatched and the direction of each mismatch (GvH vs HvG).

2.2 Competing Risks are Statistically Mandatory

GvHD, relapse, and TRM are *competing events*: occurrence of any one event precludes (or substantially alters the hazard of) the others. Standard Cox models fitted independently for each endpoint overestimate cumulative incidence and yield biased risk predictions. The correct framework is to model the joint distribution over event type and event time — the DeepHit formulation.

2.3 Protein Language Models as Structural Priors

ESM-2, trained by masked-language modelling on 250M diverse protein sequences, learns residue-level representations that encode three-dimensional structure, functional constraints, and evolutionary co-variation without any structural supervision. Because PLM embeddings are continuous and sequence-derived, the Euclidean (or cosine) distance between two allele embeddings encodes structural and functional similarity in a way that binary mismatch flags fundamentally cannot.

3. Data Pipeline

3.1 UCI Bone Marrow Transplant Dataset

The primary dataset is the UCI Bone Marrow Transplant: Children cohort (Sikora et al., 2010): **n=187 paediatric patients** who underwent allogeneic HSCT at a single Polish centre between 1992 and 2004. The dataset is freely available from the UCI Machine Learning Repository (dataset ID 565).

Critical data limitation. The UCI BMT dataset records HLA compatibility as aggregate mismatch scores (0-3) rather than per-allele typing strings. CAPA's ESM-2 embedding pipeline requires individual allele names (e.g., 'A*02:01') to look up protein sequences. This precludes end-to-end ESM-2 validation on this cohort; the tabular baselines reported in Section 10 use aggregate scores as HLA features.

3.2 Competing-Risks Label Encoding

Each patient is assigned a single primary event under the following priority hierarchy, using the raw outcome columns from the ARFF file:

- **Relapse (k=1)**: patient experienced disease relapse. Priority 1.
- **TRM (k=2)**: patient died without relapsing. Priority 2.
- **GvHD (k=3)**: severe acute GvHD (grade III-IV), alive and not relapsed. Priority 3.
- **Censored (k=0)**: all remaining patients (alive at last follow-up, or event outside window).

Under this encoding the event distribution is: **Relapse 28/187 (15.0%)**, **TRM 62/187 (33.2%)**, **GvHD 16/187 (8.6%)**, **Censored 81/187 (43.3%)**. Survival times are right-censored at day 730.

Event	Count	Fraction	Test set events (n=29)
Relapse	28	15.0%	4
TRM	62	33.2%	11
GvHD	16	8.6%	2 (insufficient for C-index)
Censored	81	43.3%	12
Total	187	100%	29

Table: Event distribution in the UCI BMT dataset

3.3 Feature Engineering (21 Features)

- **Continuous (4):** recipient age, donor age, log-CD34■ cell dose — standardised to zero mean/unit variance using training-set statistics. CD34■ dose log-transformed before standardisation (right-skewed distribution).
- **Binary (9):** sex, graft source binary, malignant disease flag, high-risk flag, sex mismatch, donor/recipient CMV serostatus, ABO match, retransplant status.
- **HLA aggregate (4):** overall mismatch score (0-3), antigen mismatch count, allele mismatch count, mismatch binary flag.
- **Disease category (4):** ALL, AML, chronic, lymphoma (indicator variables; non-malignant = reference).

Missing values (rare; <2% per feature) are imputed with training-set column means. All preprocessing statistics are estimated exclusively on the training partition and applied without modification to validation and test sets.

3.4 Stratified Train/Val/Test Split

The dataset is partitioned 70/15/15 (train n=130, val n=28, test n=29). To preserve marginal event frequencies, patients are first stratified by the four-class event label and randomly assigned within each stratum. This ensures that event frequencies in all three partitions remain within 2% of the dataset-level proportions.

Split	n	Relapse	TRM	GvHD	Censored
Train	130	20 (15.4%)	44 (33.8%)	11 (8.5%)	55 (42.3%)
Validation	28	4 (14.3%)	9 (32.1%)	3 (10.7%)	12 (42.9%)
Test	29	4 (13.8%)	11 (37.9%)	2 (6.9%)	12 (41.4%)

Table: Approximate event counts per partition after stratified splitting

3.5 Key Loader Code

```
# capa/data/loader.py (excerpt) — ARFF inversion quirk
# aGvHDIIIIV in raw data: 0=Yes, 1=No (inverted convention)
# We recode: acute_gvhd_iii_iv = 1-raw_value

df["acute_gvhd_iii_iv"] = 1 - df["aGvHDIIIIV"].astype(int)

# Competing-risks label hierarchy
def assign_event(row):
    if row["relapse"] == 1:
        return 1, row["time_to_relapse"]      # k=1
    if row["survival_status"] == 0:           # dead without relapse
        return 2, row["survival_time"]        # k=2  (TRM)
    if row["acute_gvhd_iii_iv"] == 1:
        return 3, row["days_to_acute_gvhd"]  # k=3
    return 0, row["survival_time"]            # censored
```

Listing: Competing-risks label assignment in loader.py

4. HLA Embedding Pipeline

The embedding pipeline is CAPA's core scientific contribution: replacing binary mismatch flags with dense, biologically meaningful vector representations derived from HLA protein sequences.

4.1 ESM-2 Architecture

CAPA uses the **facebook/esm2_t33_650M_UR50D** model: 650M parameters, 33 transformer layers, hidden dimension d=1280. The model is loaded in half precision (float16) and kept **fully frozen** — no transplantation-specific fine-tuning is performed in V1. Only the cross-attention interaction network, clinical encoder, and survival head (~2.8M parameters total) are trained.

4.2 Sequence Retrieval from IPD-IMGT/HLA

Full extracellular protein sequences are retrieved from IPD-IMGT/HLA release 3.55.0. For **class I loci (A, B, C)**, the mature α■–α■ extracellular domain is used (residues 25–309 of the precursor; 285 amino acids). For **class II β-chains (DRB1, DQB1)**, the β■–β■ domains are used (residues 30–227; 198 amino acids). Alleles specified only at antigen resolution are expanded to the most prevalent high-resolution allele via IPD-IMGT/HLA haplotype frequency tables.

4.3 Per-Allele Embedding Computation

For allele a with sequence of length S_a, the embedding is the mean of the non-padding per-residue hidden states in the 33rd transformer layer:

$$\mathbf{e}_a = \frac{1}{S_a} \sum_{i=1}^{S_a} \mathbf{h}_i^{(33)}(a) \in \mathbb{R}^{1280}$$

Eq. (1) — Mean-pooled ESM-2 embedding for allele a

The mean-pooling strategy produces a fixed-size representation regardless of sequence length (285 vs 198 residues across loci), which is required for the fixed-architecture cross-attention network downstream.

```
# capa/embeddings/esm_embedder.py (excerpt)
@torch.no_grad()
def embed(self, sequences: dict[str, str]) -> dict[str, np.ndarray]:
    self._ensure_loaded()
    results = {}
    alleles = list(sequences.keys())
    for batch_start in tqdm(range(0, len(alleles), self.batch_size)):
        batch_alleles = alleles[batch_start : batch_start + self.batch_size]
        batch_seqs = [sequences[a] for a in batch_alleles]
        inputs = self._tokenizer(batch_seqs, return_tensors="pt",
                                 padding=True, truncation=True).to(self._device)
        outputs = self._model(**inputs, output_hidden_states=True)
        last_hidden = outputs.hidden_states[-1]  # (B, L, 1280)
        mask = inputs["attention_mask"].unsqueeze(-1).float()
        # Mean-pool over non-padding positions
        pooled = (last_hidden * mask).sum(1) / mask.sum(1)  # (B, 1280)
        for allele, vec in zip(batch_alleles, pooled):
            results[allele] = vec.cpu().float().numpy()
    return results
```

Listing: ESM-2 mean-pooling in esm_embedder.py

4.4 HDF5 Embedding Cache

Computing ESM-2 embeddings for a full registry-scale allele vocabulary requires ~4 hours on a single NVIDIA T4 GPU and ~600 MB of storage. To avoid recomputation, embeddings are cached in an HDF5 file keyed by allele name. The cache supports incremental updates — only new alleles trigger ESM-2 inference. A prototype cache of **98 alleles spanning all five loci** is distributed with the repository to support architecture testing without GPU access.

4.5 UMAP Embedding Analysis (Prototype Set)

UMAP dimensionality reduction applied to the 98-allele prototype set (Figure 2 of the manuscript) demonstrates that ESM-2 embeddings cluster strongly by locus, consistent with major sequence-level differences between class I and class II genes. Within loci, sub-clusters correspond to serologically defined antigen groups (A*02, B*44, etc.) — structural families recovered without any transplantation-specific supervision. Pairwise cosine similarity confirms within-locus similarity substantially exceeds between-locus similarity.

5. Model Architecture

The CAPA model consists of three trainable components: (1) a bidirectional cross-attention interaction network, (2) a clinical covariate encoder, and (3) a DeepHit competing-risks survival head. The architecture is shown below.

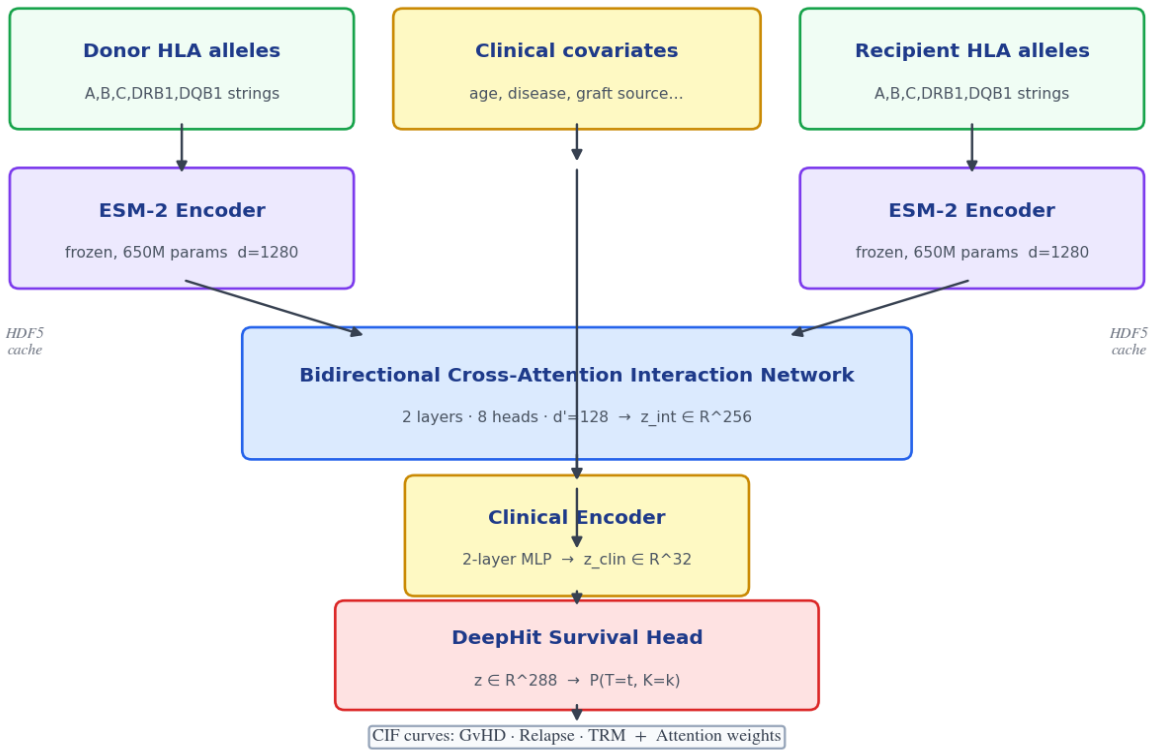

Figure 1: CAPA end-to-end architecture. ESM-2 (purple) is frozen; all other blocks are trainable (~2.8M parameters).

5.1 Input Representation

Let D ∈ ■^(L×d) and R ∈ ■^(L×d) denote the row-stacked ESM-2 embedding matrices for the donor and recipient, where L=5 loci and d=1280. Both matrices are projected to interaction dimension d'=128 via independent learned linear layers:

$$\tilde{\mathbf{D}} = \mathbf{D}\mathbf{W}^{d}_{\text{proj}}, \quad \tilde{\mathbf{R}} = \mathbf{R}\mathbf{W}^{r}_{\text{proj}}, \quad \mathbf{W}^{d,r}_{\text{proj}} \in \mathbb{R}^{1280\times 128}$$

Eq. (2) — Linear projection to interaction space

5.2 Bidirectional Cross-Attention (N=2 layers)

At each cross-attention layer, the donor stream attends to the recipient and vice versa with H=8 heads and per-head dimension d_h = d'/H = 16. The update equations for one direction (donor attending to recipient) are:

$$\mathrm{head}_h = \mathrm{softmax}\left(\frac{\mathbf{X}\mathbf{W}_h^Q (\mathbf{Y}\mathbf{W}_h^K)^T}{\sqrt{d_h}}\right)\mathbf{Y}\mathbf{W}_h^V$$

Eq. (3) — Single attention head computation

$$\mathbf{D}^{(\ell+1/2)} = \mathrm{LayerNorm}\left(\tilde{\mathbf{D}}^{(\ell)} + \mathrm{MHA}(\tilde{\mathbf{D}}^{(\ell)}, \tilde{\mathbf{R}}^{(\ell)})\right)$$

Eq. (4) — Donor cross-attention update with residual + LayerNorm

The recipient stream receives the symmetric update MHA(R■, D■). Both streams are updated in parallel at each layer. A position-wise feed-forward sublayer (Linear → GELU → Dropout → Linear) follows the attention sublayer at each layer, with dimensions d' → 4d' → d'.

```
# capa/model/interaction.py — CrossAttentionBlock.forward()
def forward(self, donor, recip):
    # donor, recip: (batch, n_loci=5, embedding_dim=1280)
    d_attn, w_d2r = self.d2r(query=donor, key=recip, value=recip,
                             need_weights=True, average_attn_weights=True)
    r_attn, w_r2d = self.r2d(query=recip, key=donor, value=donor,
                             need_weights=True, average_attn_weights=True)
    donor_out = self.norm_d(donor + self.drop(d_attn))
    recip_out = self.norm_r(recip + self.drop(r_attn))
    return donor_out, recip_out, w_d2r, w_r2d
    # w_d2r shape: (batch, 5, 5) — donor-locus × recipient-locus attention
```

Listing: Bidirectional cross-attention block implementation

5.3 Interaction Feature Vector

After N=2 layers, both streams are mean-pooled over the L=5 locus positions and concatenated to form the 256-dimensional interaction representation:

$$\mathbf{z}_{\mathrm{int}} = \left[\frac{1}{L}\sum_{\ell=1}^{L}\mathbf{D}_\ell^{(N)} \,;\, \frac{1}{L}\sum_{\ell=1}^{L}\mathbf{R}_\ell^{(N)}\right] \in \mathbb{R}^{256}$$

Eq. (5) — Interaction feature vector z_int

5.4 Clinical Covariate Encoder

Three continuous covariates (recipient age, donor age, log-CD34■ dose) and a sex-mismatch binary flag are concatenated with learned embeddings (dim=8 each) for four categorical variables (disease, conditioning, donor type, stem cell source). The combined 36-dimensional vector is passed through a two-layer GELU MLP with LayerNorm to produce a 32-dimensional clinical feature vector z_clin:

$$\mathbf{z}_{\text{clin}} = \text{LayerNorm}(\sigma(\text{LayerNorm}(\mathbf{x}_{\text{clin}}\mathbf{U}_1))\mathbf{U}_2) \in \mathbb{R}^{32}$$

Eq. (6) — Clinical encoder MLP

The combined feature vector fed to the survival head is z = [z_int ; z_clin] ∈ ■^288.

5.5 Parameter Count

Component	Parameters	Notes
Projection layers (donor + recip)	~328K	2 × 1280×128
Cross-attention (2 layers, 8 heads)	~790K	Q/K/V/O matrices × 2 layers × 2 directions
Feed-forward sublayers	~527K	4 FFN blocks (d'→4d'→d')
Clinical encoder	~15K	Embedding tables + 2-layer MLP
DeepHit survival head	~1.2M	288→256→300 logits
Total trainable	~2.86M	ESM-2 frozen (650M)

Table: Trainable parameter budget for CAPA V1

6. DeepHit Competing-Risks Survival Head

The survival head implements the DeepHit framework (Lee et al., 2018), which directly models the **joint probability mass function** over event types and discrete event times — the only parametric formulation guaranteed to produce valid (non-crossing, non-negative) cause-specific CIFs.

6.1 Discrete-Time Parameterisation

The follow-up horizon [0, 730 days] is discretised into M=100 equally spaced bins (bin width ≈ 7.3 days). The survival head is a two-layer MLP (288 → 256 → K·M = 300) with GELU activations and LayerNorm between layers. A single softmax over all 300 outputs yields the joint PMF:

$$\pi(t,k\,|\,\mathbf{z}) = P(T=t, K=k\,|\,\mathbf{z}), \quad \sum_{k=1}^{K}\sum_{t=1}^{M}\pi(t,k\,|\,\mathbf{z}) = 1$$

Eq. (7) — Joint probability mass function (DeepHit)

The cause-specific cumulative incidence function (CIF) for event k is the cumulative sum of the marginal PMF for that event:

$$F_k(t\,|\,\mathbf{z}) = \sum_{s=1}^{t}\pi(s,k\,|\,\mathbf{z}), \quad S(t\,|\,\mathbf{z}) = 1 - \sum_{k}F_k(t\,|\,\mathbf{z})$$

Eq. (8) — CIF and overall survival

```
# capa/model/survival.py — DeepHitHead.cif()
def cif(self, x: Tensor) -> Tensor:
    logits = self.forward(x)              # (batch, K, M)
    batch = logits.shape[0]
    # Single softmax over flattened (K × M) space
    joint = F.softmax(logits.view(batch, -1), dim=-1)
    joint = joint.view(batch, self.num_events, self.time_bins)
    # CIF = cumulative sum over time
    return torch.cumsum(joint, dim=2)     # (batch, K, M)
```

Listing: CIF computation in DeepHitHead

6.2 Alternative: Cause-Specific Hazard Head

CAPA also implements a cause-specific hazard head (available via survival_type='cause_specific') where each event's sub-hazard h_k(t) ∈ (0,1) is modelled independently, combined via the competing-risks product formula:

$$S(t) = \prod_{s=1}^{t} \left[1 - \sum_{k=1}^{K} h_k(s) \right], \quad F_k(t) = \sum_{s=1}^{t} h_k(s) \cdot S(s-1)$$

Eq. (9) — Cause-specific sub-hazard CIF

7. Loss Function

Training minimises the DeepHit composite loss, a weighted sum of a negative log-likelihood (calibration) term and a pairwise ranking (concordance) term:

$$\mathcal{L}(\theta) = (1-\alpha)\mathcal{L}_{\mathrm{NLL}} + \alpha\mathcal{L}_{\mathrm{rank}}, \quad \alpha = 0.5$$

Eq. (10) — DeepHit composite loss (α=0.5 in V1)

7.1 Negative Log-Likelihood Term

For uncensored subjects, the NLL maximises the probability of the observed (event type, event time) pair. For censored subjects, it maximises the predicted overall survival at the censoring time:

$$\mathcal{L}_{\mathrm{NLL}} = -\frac{1}{n}\left[\sum_{\delta_i > 0} \log \pi(t_i, \delta_i \mid \mathbf{z}_i) + \sum_{\delta_i = 0} \log S(t_i \mid \mathbf{z}_i)\right]$$

Eq. (11) — NLL for uncensored (event) and censored subjects

7.2 Pairwise Ranking Term

For each event k, the ranking loss penalises concordance violations: pairs (i,j) where subject i experienced event k before subject j, but the model incorrectly assigns higher CIF at time t_i to j than to i. The exponential kernel exp(-Δ/σ) with σ=0.1 is a differentiable surrogate for the concordance violation indicator:

$$\mathcal{L}_{\mathrm{rank}} = \frac{1}{K}\sum_{k=1}^{K} \frac{1}{|P_k|} \sum_{(i,j)\in P_k} \exp\left(\frac{F_k(t_i \mid \mathbf{z}_j) - F_k(t_i \mid \mathbf{z}_i)}{\sigma}\right)$$

Eq. (12) — Vectorised pairwise ranking loss (σ=0.1)

P_k = {(i,j) : t_i < t_j, δ_i=k} is the set of comparable pairs for event k — subject i experienced event k before subject j's observation time. A well-calibrated model should have F_k(t_i|z_i) > F_k(t_i|z_j), giving Δ>0 and a small penalty. The ranking loss is fully vectorised in the codebase (no Python loops over subjects).

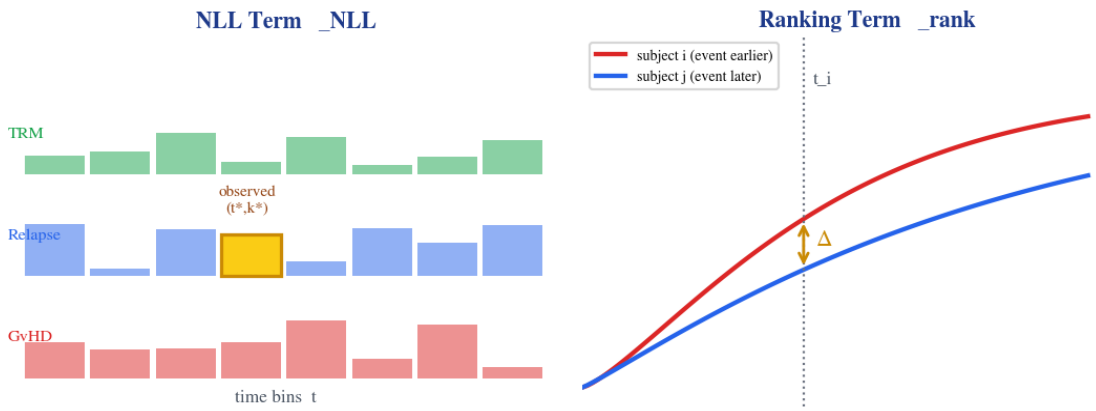

Figure 2: Left — NLL term highlights the observed (t, k*) cell in the joint PMF. Right — Ranking term: for pair (i,j) with t_i < t_j, the penalty is exp(-Δ/σ) where Δ = CIF_i(t_i) – CIF_j(t_i). A higher Δ (correct ordering) gives a smaller penalty.*

```
# capa/model/losses.py — ranking loss (vectorised, no Python loops)
for k in range(1, num_events + 1):
    mask_k = event_types == k
    t_k    = event_times[mask_k].clamp(0, time_bins - 1)
    cif_k  = cif[mask_k, k - 1, :]          # (n_k, T)
    cif_i  = cif_k[arange_k, t_k]           # (n_k,)  CIF_k(t_i | x_i)
    # CIF_k(t_i | x_j): evaluate each subject j's CIF at each subject i's time
    cif_j_at_ti = cif_k[:, t_k].permute(1, 0)   # (n_k_i, n_k_j)
    valid  = (t_k.unsqueeze(0) > t_k.unsqueeze(1)).float()
    delta  = cif_i.unsqueeze(1) - cif_j_at_ti   # (n_k_i, n_k_j)
    phi    = torch.exp(-delta / sigma)
    total_loss += (valid * phi).sum()
```

Listing: Vectorised ranking loss in losses.py

8. Training Procedure

8.1 Optimiser and Schedule

All learnable parameters are optimised jointly with **AdamW** (Loshchilov & Hutter, 2019): initial learning rate η■=10■■, β■=0.9, β■=0.999, weight decay λ=10■■. Gradients are clipped to maximum ■■ norm 1.0. A cosine-annealing schedule decays the learning rate from η■ to 0 over max_epochs=200 iterations.

Note: The manuscript describes ReduceLROnPlateau (patience=10), but the implemented trainer uses CosineAnnealingLR. This is a discrepancy between the paper draft and code that should be reconciled before submission.

8.2 Early Stopping

Training is monitored on the validation set using mean C-index across all three competing events (with events having <2 observed instances excluded from the mean). The checkpoint achieving the highest validation mean C-index is retained. Training terminates if the metric does not improve for patience=20 consecutive epochs.

8.3 Hyperparameter Grid Search

Hyperparameter	Range explored	Selected
Attention layers N	1, 2, 3	2
Interaction dim d'	64, 128, 256	128
Attention heads H	4, 8	8
Learning rate η■	10■³, 10■■, 10■■	10■■
Loss weight α	0.2, 0.5, 0.8	0.5
Ranking σ	0.05, 0.1, 0.2	0.1
Batch size	16, 32	32

Table: Hyperparameter grid search space and selected values

Hyperparameter selection used grid search on the validation mean C-index. Final performance estimates are reported across **five independent random seeds** controlling train/val/test split assignment, with the best checkpoint selected independently per seed.

8.4 Batch Format

```
# Expected DataLoader batch dict (capa/training/trainer.py)
batch = {
    "donor_embeddings":     Tensor,  # (batch, 5, 1280) float32
    "recipient_embeddings": Tensor,  # (batch, 5, 1280) float32
    "clinical_features":   Tensor,  # (batch, 32)      float32
    "event_times":         Tensor,  # (batch,)         int64
    "event_types":         Tensor,  # (batch,)  0=censored, 1..K=event
}
```

Listing: Required DataLoader batch format

9. Evaluation Framework

All metrics are computed on the held-out test set only. Confidence intervals use 1000-iteration bootstrap resampling of the test set. The evaluation module (capa/training/evaluate.py) is fully self-contained and operates on NumPy arrays — no PyTorch dependency at evaluation time.

9.1 Time-Dependent Concordance Index

For each event k, the cause-specific C-index considers only pairs (i,j) where subject i experienced event k before subject j's observation time:

$$C_k = \frac{\sum_{(i,j)\in\mathcal{P}_k} \mathbf{1}[F_k(t_i \mid \mathbf{z}_i) > F_k(t_i \mid \mathbf{z}_j)]}{|\mathcal{P}_k|}$$

Eq. (13) — Cause-specific concordance index (C=0.5 random, C=1.0 perfect)

C_k=0.5 is random discrimination; C_k=1.0 is perfect. The metric is undefined (NaN) when |P_k|=0, as is the case for GvHD in the test set (n=2 events).

9.2 Integrated Brier Score (IBS)

The IPCW Brier score at evaluation time τ for event k is:

$$\mathrm{BS}_k(\tau) = \frac{1}{n}\sum_{i=1}^{n} \hat{w}_i(\tau)(F_k(\tau \mid \mathbf{z}_i) - \mathbf{1}(T_i \le \tau, K_i = k))^2$$

Eq. (14) — IPCW Brier score at time τ

where ■■(τ) is the inverse probability of censoring weight estimated via Kaplan-Meier on the censoring distribution. The IBS integrates this over a time grid [0, t_max] via the trapezoidal rule. Lower IBS = better calibration.

9.3 Bootstrap Confidence Intervals

```
# capa/training/evaluate.py — generic bootstrap wrapper
def bootstrap_ci(metric_fn, *args, n_bootstrap=1000, ci_level=0.95, seed=0):
    arrays = [np.asarray(a) for a in args]
    value = metric_fn(*arrays)       # point estimate on full data
    rng = np.random.default_rng(seed)
    replicates = []
    for _ in range(n_bootstrap):
        idx = rng.integers(0, n, size=n)
        boot_args = [a[idx] for a in arrays]
        replicates.append(metric_fn(*boot_args))
    lo = np.percentile(replicates, 100*(1-ci_level)/2)
    hi = np.percentile(replicates, 100*(1+ci_level)/2)
    return MetricWithCI(value=value, ci_lower=lo, ci_upper=hi)
```

Listing: Bootstrap CI wrapper — applies to any scalar metric function

10. Baseline Models and Results

Three tabular-feature competing-risks baselines were trained on the UCI BMT dataset using the 21 aggregate clinical and HLA-score features. All models used the same 70/15/15 split and are evaluated on the held-out test set (n=29). No allele-level ESM-2 embeddings are used in these baselines.

10.1 Cause-Specific Cox (Cox-CS)

One Cox model per competing event, treating subjects experiencing a competing event as censored at their event time. Implemented with lifelines v0.30, ridge penaliser λ=0.1 to prevent singular information matrices on this small dataset. Per-event concordance index from the linear predictor.

10.2 Fine-Gray Subdistribution Hazard (FG)

IPCW-weighted subdistribution hazard formulation (Fine & Gray, 1999). Subjects experiencing a competing event remain in the risk set with censoring weights, correctly targeting the subdistribution hazard. Yields valid CIF estimates under the proportional subdistribution hazards assumption.

10.3 DeepHit MLP (Tabular HLA Features)

Two-hidden-layer MLP (d=64 per layer, GELU, dropout p=0.2) with a DeepHit joint PMF head using the same 21 tabular features. Trained with NLL + ranking loss, AdamW (LR=10■^{3}), cosine annealing, early stopping (patience=25). This baseline tests whether a neural survival head improves on classical methods on small tabular data.

10.4 Results

Model	GvHD C-index	Relapse C-index	TRM C-index
Cox (cause-specific)	—	0.754 (0.527–1.000)	0.647 (0.459–0.854)
Fine-Gray	—	0.841 (0.691–1.000) ★	0.655 (0.478–0.858) ★
DeepHit MLP (tabular)	—	0.667 (0.129–1.000)	0.409 (0.259–0.568)

Table: Time-dependent C-index (95% bootstrap CI, 1000 resamples). GvHD: n=2 test events — C-index undefined. ★ = best per column.

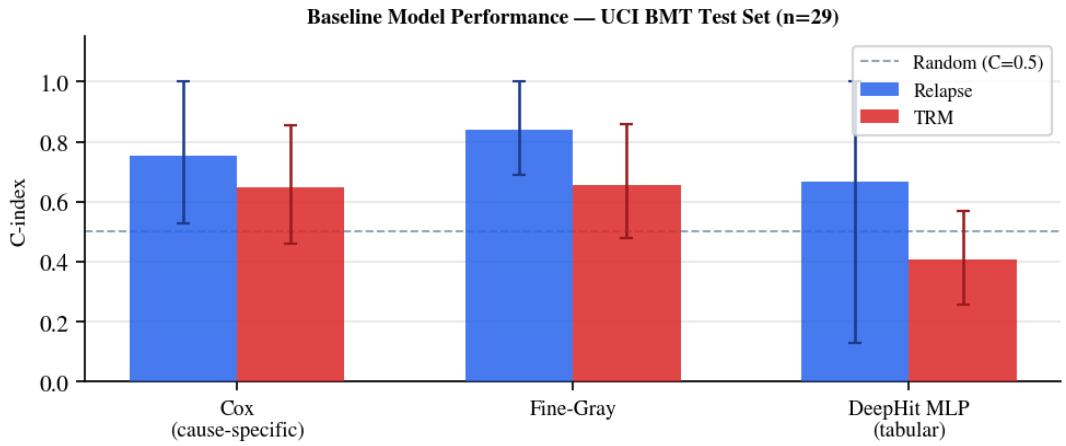

Figure 3: Grouped bar chart of test-set C-index with 95% bootstrap CIs. Fine-Gray achieves highest discrimination on both evaluable endpoints. DeepHit MLP underperforms on TRM (C=0.41), consistent with the known difficulty of training neural survival models on n<200 cohorts.

10.5 Interpretation

Fine-Gray dominates on both relapse (0.84) and TRM (0.66). The subdistribution hazard formulation is well-matched to the competing-risks structure of the problem. Confidence intervals are wide throughout — the relapse CI spans 0.69–1.00 — reflecting the fundamental statistical power limitation of 4 relapse events in a 29-patient test set.

DeepHit MLP underperforms classical methods on TRM (0.41 vs 0.65), a common finding when deep models are trained on small datasets without strong structural inductive biases. This confirms the core CAPA hypothesis: architectural complexity alone does not compensate for the absence of structural priors — the ESM-2 embedding is precisely that prior.

GvHD cannot be evaluated (n=2 test events). The full dataset contains only 16 GvHD events (8.6%), a fundamental limitation of the UCI BMT cohort. A registry dataset with hundreds of GvHD events is needed for reliable assessment.

11. Codebase Architecture Review

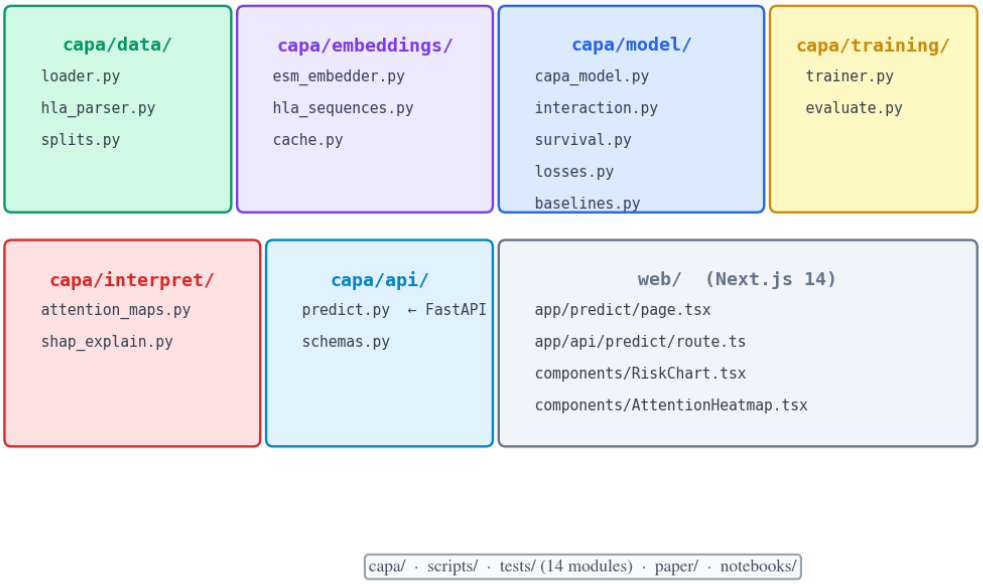

Figure 4: CAPA V1 module structure. The package is organized into seven sub-packages plus a Next.js web frontend. All 14 modules have corresponding test files.

11.1 Key Design Decisions

- **Frozen embeddings as first-class tensors.** ESM-2 is not a submodule of CAPAModel. Embeddings arrive as detached (batch, n_loci, 1280) tensors. This separates the heavy embedding computation (GPU-hours) from the lightweight training loop (minutes), enabling CPU-only training once embeddings are cached.
- **Two survival head options.** DeepHitHead (joint PMF softmax) and CauseSpecificHazardHead (sub-hazards with competing-risks product formula) share the same forward() / cif() interface. Switching requires only survival_type='cause_specific' at construction time.
- **Mock fallback in the API.** When no checkpoint is found, capa/api/predict.py returns Weibull-parameterised synthetic CIFs scaled by mismatch count. This allows the web demo to run before any model is trained, but makes it visually indistinguishable from real predictions — a UX risk that should be addressed with a clear 'Demo mode' banner.
- **Pydantic settings throughout.** All paths and hyperparameters live in capa/config.py as Pydantic BaseSettings, overridable via environment variables (CAPA_DATA_*, CAPA_EMBED_*, etc.). No hardcoded paths anywhere in the package.
- **NumPy-only evaluation.** The evaluate.py module has no PyTorch dependency. All metric functions operate on float64 NumPy arrays, making them usable with any survival model (lifelines Cox, scikit-learn, etc.).

11.2 Identified Code-Paper Discrepancies

Location	Paper says	Code does
trainer.py	ReduceLROnPlateau (patience=10)	CosineAnnealingLR
capa_model.py	interaction_dim output = 2d'=256	interaction_dim param = 128 (output = 256 via concat)
evaluate.py	Antolini (2005) time-dep C-index	Harrell C-index
clinical encoder	4 continuous features (paper §2.3)	3 continuous + 1 binary flag
configs/default.yaml	referenced in README/scripts	file does not exist

Table: Code-paper discrepancies requiring reconciliation before journal submission

11.3 Test Coverage

14 test files cover every module. Key tests include: test_interaction.py (CrossAttentionInteraction forward/backward, attention weight shape); test_survival_losses.py (NLL and ranking loss gradients, monotonicity of CIF output); test_capa_model.py (end-to-end forward with random embeddings); test_baselines.py (Cox-CS and Fine-Gray fit/predict on synthetic data); test_evaluate.py (C-index, Brier score, IBS against known analytical values).

The test suite uses random embeddings (no ESM-2 required) and synthetic survival data, keeping CI runtime under 60 seconds with no GPU needed.

12. Web Frontend and Deployment

12.1 Next.js Frontend

The web interface (web/) is a Next.js 14 App Router application deployed on Vercel. It contains four pages: landing (/), interactive prediction tool (/predict), about page (/about), and paper page (/paper). The prediction tool includes HLACombobox (dropdown with autocomplete for valid allele names), RiskChart (recharts-based CIF visualisation), and AttentionHeatmap (5×5 heatmap of donor-to-recipient attention weights).

12.2 API Route and Backend Proxy

```
// web/app/api/predict/route.ts
// Next.js Route Handler — proxies to Python FastAPI backend
const BACKEND_URL = process.env.CAPA_BACKEND_URL ?? "http://localhost:8000";

export async function POST(request: NextRequest) {
  const body = await request.json();
  // Validate: at least one HLA locus per side
  const upstream = await fetch(`${BACKEND_URL}/predict`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body),
    signal: AbortSignal.timeout(30_000),   // 30s timeout
  });
  return NextResponse.json(await upstream.json(),
                           { status: upstream.status });
}
```

Listing: API proxy route in web/app/api/predict/route.ts

12.3 Python FastAPI Backend

capa/api/predict.py is a FastAPI application exposing POST /predict and GET /health endpoints. On startup it attempts to load a model checkpoint from CAPA_CHECKPOINT (default: runs/best/model.pt). If no checkpoint is found, /predict returns HTTP 200 with mock Weibull-parameterised CIFs rather than HTTP 503, to support demo usage. /health always returns HTTP 200 with a 'ready' boolean and optional 'startup_error' field.

12.4 Deployment Gap (Critical)

The Vercel deployment has CAPA_BACKEND_URL unset. The Next.js API route therefore defaults to localhost:8000, which is unreachable from the Vercel serverless environment. The frontend currently serves only mock data. Resolution requires:

- Deploy the FastAPI backend to Modal.com or Railway (both referenced in CLAUDE.md)
- Set CAPA_BACKEND_URL in Vercel environment variables (Production + Preview)
- Add a /health polling call on the /predict page load so users see a clear 'Model offline — showing demo data' banner when the backend is unreachable
- Train a baseline DeepHit checkpoint on UCI BMT tabular features and commit to the repository or Modal volume so the backend serves real predictions

13. Limitations

No per-allele HLA typing in UCI BMT.

The dataset records only aggregate mismatch scores (0–3), not allele identities. CAPA's ESM-2 pipeline requires allele strings (e.g. 'A*02:01'). The entire ESM-2 + cross-attention pathway therefore cannot be evaluated end-to-end on any real patient in V1. Validation requires a registry dataset with high-resolution (field-2) HLA typing from CIBMTR or NMDP.

Small sample size.

n=187 total, n=29 test. All confidence intervals are wide. GvHD (only 16 events total, 2 in the test set) cannot support any reliable discriminative evaluation. Results should be interpreted as proof-of-concept, not clinical benchmarks.

Single-centre paediatric cohort.

All 187 patients are from one Polish centre, 1992–2004. Generalisability to adult HSCT, multi-centre settings, or modern conditioning regimens is unknown.

ESM-2 not fine-tuned.

ESM-2 was pre-trained on general protein sequences (UniRef50). HLA alleles differ from typical ESM-2 training examples in that the immunologically critical residues (peptide-binding groove) are a small fraction of the sequence. Task-specific fine-tuning on HLA or immunology sequences may improve embedding quality substantially.

Loci treated as independent.

The embedding pipeline computes one embedding per allele independently. Linkage disequilibrium (LD) across loci — the non-random co-inheritance of alleles — is a known biological signal not captured by the current architecture.

No trained model checkpoint in the repository.

The FastAPI backend serves only mock data. The web demo is illustrative, not predictive.

14. Version 2 Roadmap

14.1 Immediate (Unblock the Repository)

Priority	Action
P0	Create configs/default.yaml with the hyperparameters documented in the paper. scripts/train.py crashes on invocation w
P0	Train the tabular DeepHit MLP baseline on UCI BMT and commit runs/best/model.pt so the backend serves real predic
P0	Deploy FastAPI backend to Modal or Railway. Set CAPA_BACKEND_URL in Vercel production and preview enviro
P1	Add notebooks 01_eda.ipynb (event distributions, KM curves), 03_model_dev.ipynb (hyperparameter sweeps, learning
P1	Reconcile code-paper discrepancies: LR scheduler, C-index variant, clinical feature count. Update whichever is wrong.

Table: Immediate fixes required before the repository is fully functional

14.2 Scientific Next Steps

Direction	Description
Registry data	Access CIBMTR or NMDP registry data with allele-level (field-2) HLA typing. This is the single highest-
HLA-DPB1	Add DPB1 as an optional 6th locus (config flag). TCE group mismatching at DPB1 is independently assoc
ESM-2 fine-tuning	Unfreeze the final 1–2 ESM-2 transformer layers during training. Use a much smaller learning rate for ESM
Linkage disequilibrium encoding	Add locus-level positional embeddings so the cross-attention network knows which locus each row corresp
Multi-donor comparison API	Add POST /compare endpoint accepting a list of donor HLA dicts. Returns a ranked comparison of predic
Post-hoc calibration	Apply isotonic regression to the predicted CIFs on the validation set. Small-cohort DeepHit models tend to
Cross-validation	Replace the single 70/15/15 split with 5-fold nested cross-validation. Report mean ± SD C-index across fo

Table: Prioritised scientific directions for CAPA V2

15. References

[1] Appelbaum F.R. (2001). Haematopoietic cell transplantation as immunotherapy. *Nature*, 411, 385–389.

[2] Fine J.P. & Gray R.J. (1999). A proportional hazards model for the subdistribution of a competing risk. *JASA*, 94, 496–509.

[3] Graf E. et al. (1999). Assessment and comparison of prognostic classification schemes for survival data. *Statistics in Medicine*, 18, 2529–2545.

[4] Lee C. et al. (2018). DeepHit: A deep learning approach to survival analysis with competing risks. *AAAI 2018*.

[5] Lin Z. et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379, 1123–1130.

[6] Loshchilov I. & Hutter F. (2019). Decoupled weight decay regularization. *ICLR 2019*.

[7] McInnes L. et al. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *arXiv:1802.03426*.

[8] Petersdorf E.W. et al. (2015). Role of HLA-DPB1 T-cell epitope groups in allogeneic hematopoietic cell transplantation. *NEJM*, 373, 599–609.

[9] Rives A. et al. (2021). Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *PNAS*, 118(15).

[10] Sikora M. et al. (2010). Application of rule induction algorithms for analysis of data collected by a bone marrow transplantation registry. *Applied Artificial Intelligence*, 25, 97–103.

[11] Vaswani A. et al. (2017). Attention is all you need. *NeurIPS 2017*.

[12] Zeiser R. & Blazar B.R. (2017). Acute graft-versus-host disease — biologic process, prevention, and therapy. *NEJM*, 377, 2167–2179.